

☐ Hide inserted text ☐ Hide deleted text

P0612R0: NB comment CH 2: volatile

Change in 1.9 [intro.execution] paragraph 8:

The least requirements on a conforming implementation are:

- ~~Access to volatile objects~~ **Accesses through volatile glvalues** are evaluated strictly according to the rules of the abstract machine.
- ...

Change in 1.9 [intro.execution] paragraph 12:

Reading an object designated by a volatile glvalue (3.10), modifying an object, calling a library I/O function, or calling a function that does any of those operations are all side effects, which are changes in the state of the execution environment. Evaluation of an expression (or a sub-expression) in general includes both value computations (including determining the identity of an object for glvalue evaluation and fetching a value previously assigned to an object for prvalue evaluation) and initiation of side effects. When a call to a library I/O function returns or an access ~~to a volatile object~~ **through a volatile glvalue** is evaluated the side effect is considered complete, even though some external actions implied by the call (such as the I/O itself) or by the volatile access may not have completed yet.

Change in 1.10.2 [intro.progress] paragraph 1:

The implementation may assume that any thread will eventually do one of the following:

- terminate,
- make a call to a library I/O function,
- ~~read or modify a volatile object~~ **perform an access through a volatile glvalue**, or
- perform a synchronization operation or an atomic operation.

[Note: ...]

Change in 1.10.2 [intro.progress] paragraph 3:

During the execution of a thread of execution, each of the following is termed an execution step:

- termination of the thread of execution,
- ~~access to a volatile object~~ **performing an access through a volatile glvalue**, or
- completion of a call to a library I/O function, a synchronization operation, or an atomic operation.

Change in 7.1.7.1 [dcl.type.cv] paragraph 6:

~~What constitutes an access to an object that has volatile-qualified type is~~ **The semantics of an access through a volatile glvalue are** implementation-defined. If an attempt is made to ~~refer to~~ **access** an object defined with a volatile-qualified type through the use of a **non-volatile** glvalue ~~with a non-volatile-qualified type~~, the behavior is undefined.

Drafting note: A volatile member in a class does not prevent the class from having a trivial copy constructor. That means, in particular, that values of such types can be passed in registers. I personally consider this incorrect, but the ABI breakage inherent in changing the direction was considered too severe. See core issues 496 and 2094.